VIT

VELLORE INSTITUTE OF TECHNOLOGY

Fall Semester 2026-27

BCSE303P - OPERATING SYSTEMS LAB

PROJECT REVIEW-2 REPORT

(Mid-Progress Review)

Energy-Aware CPU Scheduling in Linux using sched_ext: Core-Compaction Scheduling for Reduced Energy Consumption

Team Member 1: Mohammed Asif A
Register Number: 24BCE2076
Team Member 2: Konasani Vaibhav
Register Number: 24BCE0419
Class Number: VL2026270103058
Slot: L31 + L32
Date: 24/09/2026

1. PROJECT TITLE & TEAM DETAILS

Item	Details
Project title	Energy-Aware CPU Scheduling in Linux using sched_ext: Core-Compaction Scheduling for Reduced Energy Consumption
Team members	Mohammed Asif A — 24BCE2076 Konasani Vaibhav — 24BCE0419
Faculty	SUCHITHRA J
Course	BCSE303P – Operating Systems Lab, Fall Semester 2026-27
Class number / Slot	VL2026270103058 / L31 + L32
School / Department	SCOPE
Review	Project Review-2 (mid-progress review). Review-1 report dated 27/08/2026.

1.1 Team Contribution

The six work packages completed so far were divided between the two team members as listed below, together with their evidence in the project repository.

Work package	Main artefacts	Member(s)
Environment: guest kernel with sched_ext, KVM test VM	vm/, scripts/, build logs	Konasani Vaibhav — 24BCE0419
Scheduler: BPF policy, controller, loader	scheduler/scx_compact.bpf.c, scx_compact.c	Mohammed Asif A — 24BCE2076
Latency guard design and diagnostics	scheduler/scx_compact.bpf.c, results/raw/vm/guard_diag_*.log	Konasani Vaibhav — 24BCE0419
Test suite and workloads	tests/vm_functional.sh, tests/dutywork.c, tests/latprobe.c	Mohammed Asif A — 24BCE2076
Experiments and statistical analysis	tests/vm_bench_*.sh, tests/analyze_*.py, results/	Konasani Vaibhav — 24BCE0419
Documentation and reports	docs/, reports/	Mohammed Asif A — 24BCE2076

2. INTRODUCTION & BACKGROUND

2.1 Brief Project Background

The CPU scheduler is the part of the operating system that decides which ready task runs on which processor and when. The default fair scheduler in current Linux is EEVDF (Earliest Eligible Virtual Deadline First) [2], [6], which aims to share CPU time fairly while keeping response times low. **sched_ext** is a Linux feature that lets a scheduling policy be written as a small BPF program and loaded while the system is running. If the BPF scheduler misbehaves, the kernel automatically switches back to the default scheduler [1]. This makes it practical to build and test a real kernel scheduling policy without rebuilding the kernel for every change.

2.2 Context of the Problem

A processor saves most of its idle power when a core stays idle long enough to enter a deep sleep (idle) state [7]. On a machine with identical cores, EEVDF spreads ready tasks over idle CPUs to favour throughput and response time, so even a light load keeps many cores briefly busy. Linux Energy Aware Scheduling applies only to processors with different core types and an energy model, and it does not support SMT (two hardware threads per core) [3], so typical x86 laptops get no energy-driven packing of work. **Core compaction** packs a light load onto a small, load-dependent set of CPUs and leaves the others idle. The idea is not new: scx_lavd already implements it together with many other mechanisms [4]. This project builds a small, transparent version of it and measures its trade-offs.

2.3 Why the Project is Relevant to Operating Systems

The project implements and evaluates a CPU-scheduling policy inside the Linux kernel. It works directly with run queues (dispatch queues), time slices, CPU selection, CPU affinity, context switches, waiting time and completion time, and kernel CPU accounting. It maps to course outcome CO2, "design scheduling algorithms to compute and compare various scheduling criteria" [5].

3. PROBLEM STATEMENT

3.1 Problem Being Addressed

Under partial load, the default scheduler keeps more CPUs active than the load requires. This limits how long CPUs can remain fully idle, which is the precondition for idle-power savings.

3.2 Existing Issue or Limitation

- EEVDF has no energy-driven packing on symmetric CPUs, and Energy Aware Scheduling does not apply to them [2], [3].
- Existing sched_ext compaction (scx_lavd) is combined with many other mechanisms, so the effect of compaction alone is hard to isolate [4].
- Packing work onto fewer CPUs makes tasks wait for each other. This project measured that queueing-latency cost directly (Section 7.4).

3.3 Scope of the Project

In scope: a sched_ext scheduler that uses a load-sized set of active CPUs, grows and shrinks that set gradually, and has an optional latency guard; correctness tests; and a repeated comparison with EEVDF on CPUs used, completion time, latency and context switches. **Out of scope:** changes to EEVDF itself, CPU frequency control, application-specific policies and production deployment. **Energy:** measuring energy requires Intel RAPL energy counters on Linux running directly on the hardware (bare metal) [8], [9]. It is part of the scope but still pending (Section 8).

4. OBJECTIVES

4.1 Main Objective

To design, implement and evaluate a load-aware core-compaction CPU scheduler for Linux using sched_ext that reduces the number of CPUs in use under partial load, and to compare its CPU-usage, performance and latency behaviour (and, on supported hardware, its energy) with the default EEVDF scheduler.

4.2 Specific Objectives and Status at Review-2

#	Specific objective	Status
1	Set up a reproducible environment with a sched_ext-enabled kernel and toolchain	Done
2	Implement core compaction: place tasks only on a small active CPU set	Done
3	Grow the set under contention and shrink it with hysteresis when load falls	Done
4	Verify correctness: clean attach/detach, no starvation of pinned tasks, no watchdog errors	Done (12/12 checks)
5	Compare with EEVDF: CPUs used, completion time, scheduling latency, context switches	Done for synthetic partial-load workloads (n = 10); compile and mixed workloads pending
6	Limit the latency cost of compaction (added after it was measured)	Done: latency guard, validated (n = 10)
7	Measure energy with Intel RAPL on bare metal	Pending: not possible in the current environment

4.3 Expected Outcome

A working, observable sched_ext scheduler, plus measured evidence of how many CPUs it frees under partial load, what it costs in completion time and tail latency, whether the latency guard reduces that cost, and (after bare-metal runs) whether energy consumption falls. No specific improvement is assumed in advance.

5. METHODOLOGY / SYSTEM DESIGN

5.1 Proposed Approach

The scheduler keeps an **active CPU set**: the first nr_active CPUs in a fixed order that fills both hardware threads of one core before moving to the next core. A woken task is placed only on an idle CPU inside this set. CPUs outside the set take no work, so they stay idle. A controller runs every 20 ms and resizes the set from the measured CPU demand, the number of waiting tasks and (optionally) the measured waiting time. The same program with compaction switched off (-n) is used as a control condition, to separate the effect of compaction from the cost of running under sched_ext at all.

5.2 System Architecture / Design

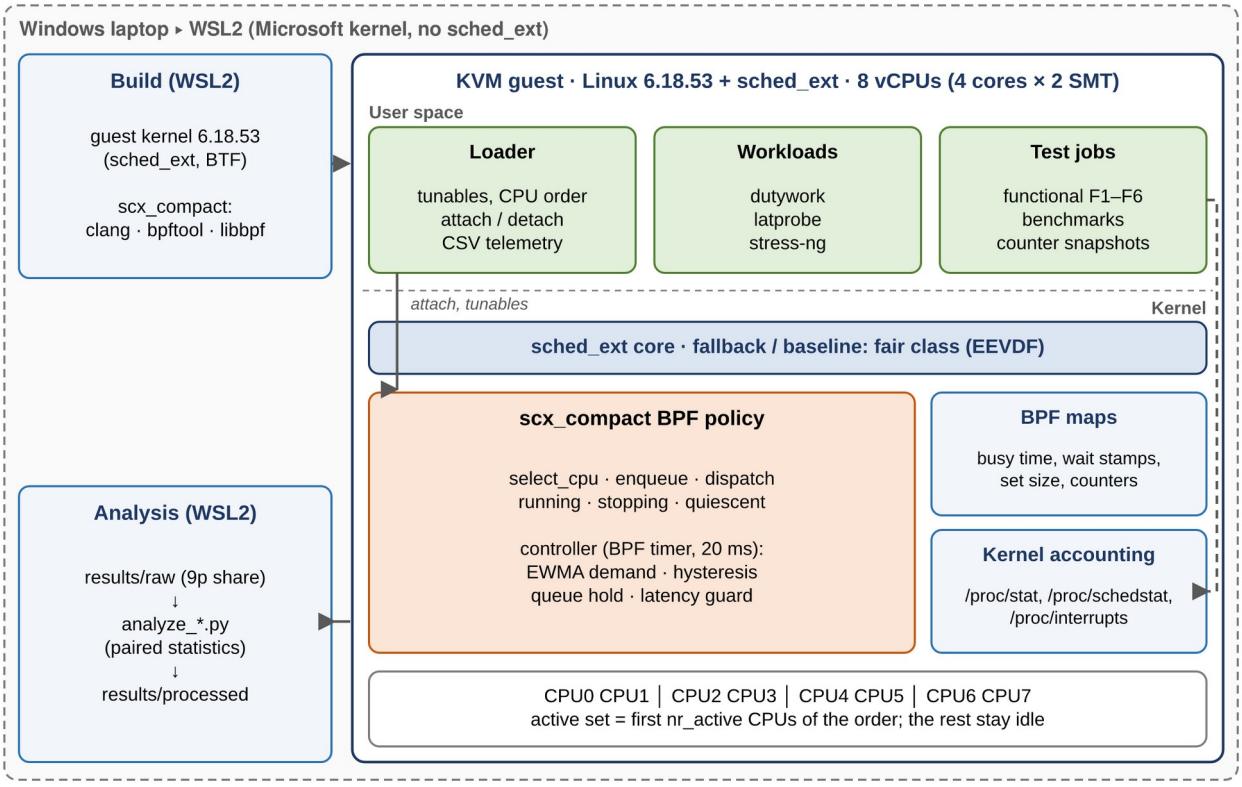
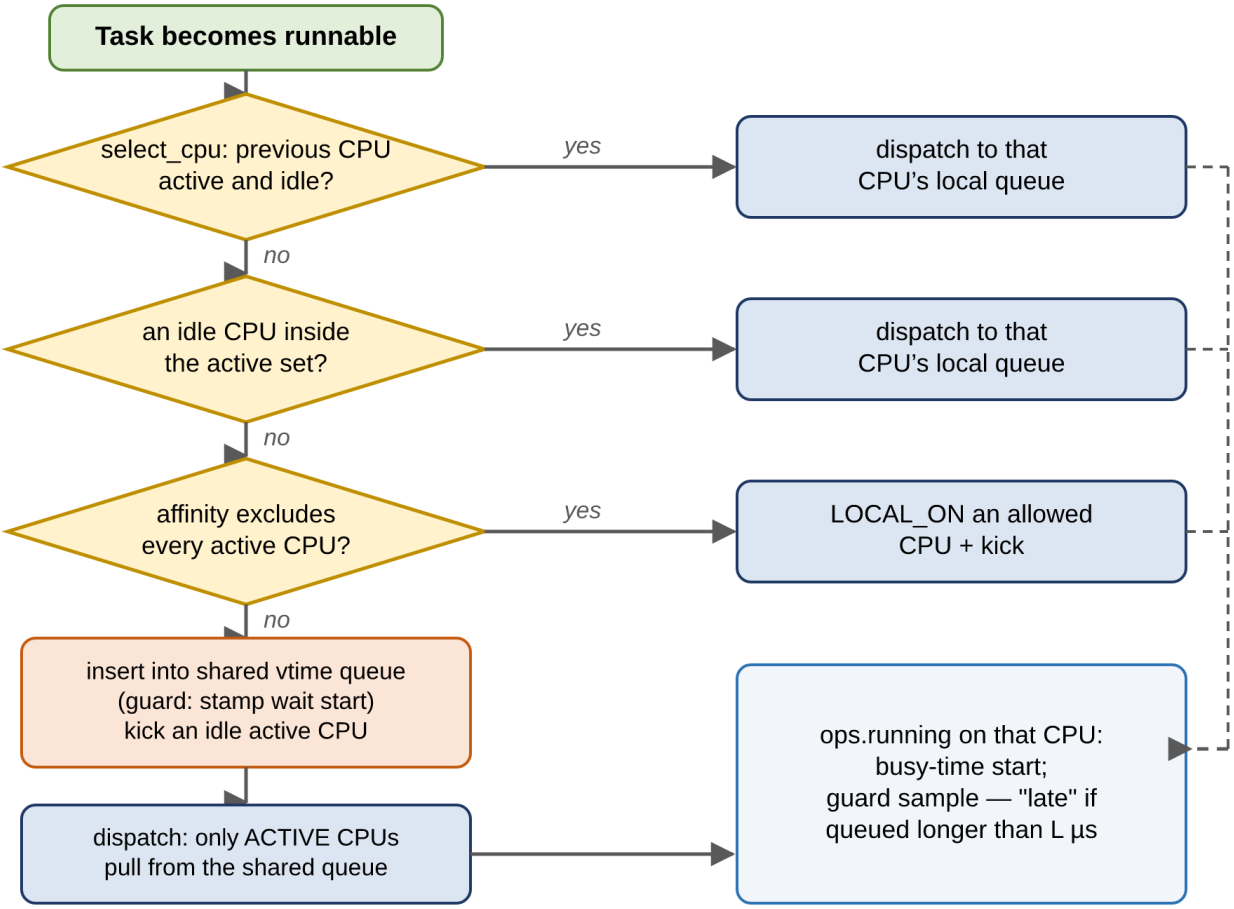


Figure 1. Implemented system architecture (build and analysis in WSL2; scheduler runs in a KVM guest with sched_ext).

The WSL2 kernel on the development laptop does not include sched_ext, so the project builds its own Linux 6.18.53 kernel with sched_ext enabled and runs every test in a virtual machine (QEMU/KVM) with 8 virtual CPUs (4 cores × 2 hardware threads). A small user program, the loader, starts the BPF scheduler, passes its settings and records its state every 200 ms. The test scripts record the kernel's CPU statistics before and after each run, and analysis scripts turn the raw data into the result tables.



CPUs outside the active set never pull shared work, so they stay idle.

Figure 2. Task placement path in scx_compact (compaction enabled).

5.3 Controller and Latency Guard

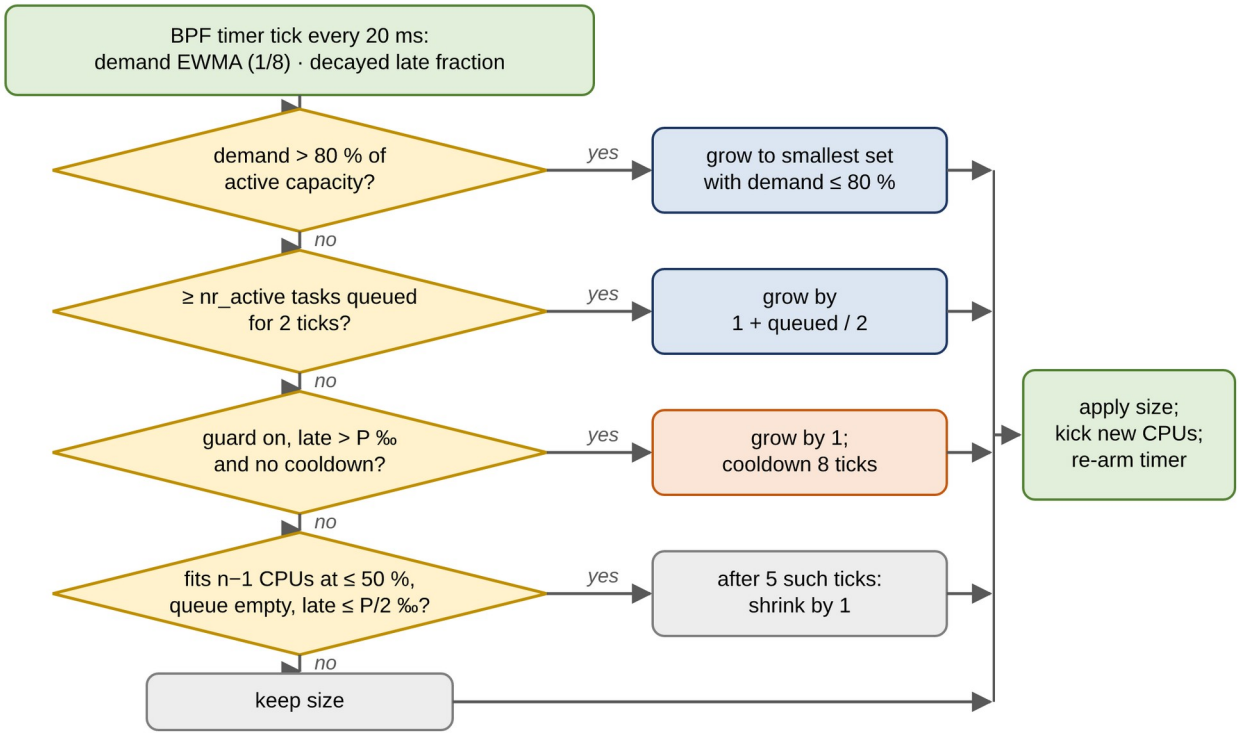


Figure 3. Controller decision per 20 ms tick; the latency-guard step is optional (-L).

- **Demand:** the total CPU busy time in each 20 ms interval, smoothed with a moving average (weight 1/8) so that short bursts do not resize the set.
- **Expand** when demand exceeds 80 % of the set's capacity, or when at least as many tasks as there are active CPUs have been waiting for 2 consecutive ticks. **Shrink** by one CPU only after the load has fitted into one CPU fewer at ≤ 50 % utilisation for 5 consecutive ticks; this slow release (hysteresis) stops the set size from oscillating.
- **Latency guard** (-L 1000 -P 10, off by default): compaction can make a task wait because every active CPU is busy. The guard measures exactly this delay, i.e. the time a task spends in the shared queue before it runs; only tasks that are allowed to run on any CPU are counted, and a task placed directly on an idle CPU counts as not delayed. If more than 1 % of recent dispatches waited longer than 1 ms, the guard adds one CPU and then pauses for 8 ticks. The set may not shrink again until fewer than 0.5 % of dispatches are late; after that, the normal slow shrinking applies.

5.4 Evaluation Methodology

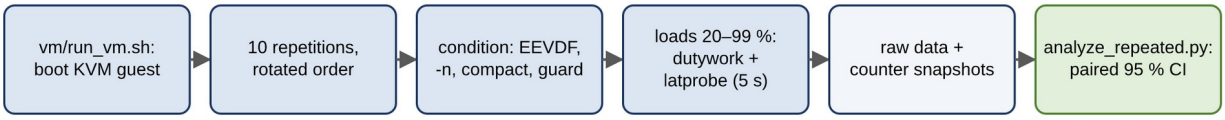


Figure 4. Repeated, interleaved evaluation pipeline.

Condition	Meaning
EEVDF	No BPF scheduler loaded: default Linux scheduling (baseline)
Compaction off (-n**)**	scx_compact with all CPUs active: cost of the sched_ext path without compaction
Compact	scx_compact with default settings (latency guard off)
Compact + guard	scx_compact -L 1000 -P 10 (p99 target: 1 ms policy-induced queueing delay)

Workload: 8 dutywork workers, each doing a fixed amount of CPU work for 20, 40, 60 or 99 % of every 20 ms period (250 periods; ideal completion time 5.0 s). Alongside them, a latprobe task wakes up every 1 ms and records how late each wake-up is; it represents an interactive task under background load. Each of the 10 repetitions runs all four conditions in a rotated order, and the scheduler is restarted every time. Measured values: CPUs used (≥ 10 % busy) and CPUs parked (< 1 % busy), active-set size, completion time, wake-up and probe latency (p95/p99), average run-queue waiting time and context switches. Each condition is compared with EEVDF **within the same repetition (paired)**, with a 95 % bootstrap confidence interval; a difference is called clear only when this interval excludes zero.

5.5 Technologies, Tools and OS Concepts Used

Category	Used in this project
Kernel / framework	Linux 6.18.53 with sched_ext; the policy is a BPF program attached through sched_ext callbacks, with a BPF timer and BPF maps (shared kernel data tables)
Languages / build	C for the scheduler, loader and workloads (BPF code compiled with clang and loaded with libbpf); Python for analysis
Environment	Windows laptop (Intel Core i7-13650HX) → WSL2 → QEMU/KVM guest
Workloads / tools	dutywork and latprobe (developed in the project), stress-ng, /proc/stat, /proc/schedstat, /proc/interrupts
OS concepts	scheduling classes, run queues / dispatch queues, fair (virtual-time) ordering, time slices, CPU affinity, idle-CPU selection, context switches, waiting vs completion time, tickless idle, kernel CPU-time accounting

5.6 Major Modules / Components

Module	Role	Location
BPF scheduling policy + controller	placement, shared queue, active-set control, latency guard	scheduler/scx_compact.bpf.c, scx_compact.h
Loader	CPU order, tunables, attach/detach, CSV telemetry	scheduler/scx_compact.c
Guest kernel and VM	sched_ext kernel build; KVM runner; guest init	vm/
Workloads	fixed-work duty cycle; 1 ms latency probe	tests/dutywork.c, tests/latprobe.c
Test and benchmark jobs	functional checks F1–F6; benchmark suites	tests/vm_*.sh
Analysis	per-run metrics, paired statistics, reports	tests/analyze_*.py → results/processed/

5.7 Justification of Design Choices

Choice	Reason
sched_ext	A real kernel scheduling policy that can be loaded and unloaded at run time, with automatic fallback to the fair scheduler on errors.
EEVDF as baseline	It is the default Linux scheduler, so it is the policy compaction must be compared against.
Compaction-off control (-n)	It separates the cost of the sched_ext path from the effect of compaction. This proved essential: the full-load latency penalty appears with -n too (Section 7.4).
Active set + hysteresis	A simple, observable rule. Slow shrinking avoids oscillation between set sizes.
Latency guard	Utilisation cannot see burstiness. The measured tail-latency cost came from the sizing rule keeping 4 CPUs for 3.1 CPUs of bursty demand.
Guard measures only policy-caused queueing delay	Some waits cannot be shortened by adding CPUs (slow wake-ups of idle virtual CPUs, per-CPU kernel threads); counting them would make the guard add CPUs uselessly. The diagnostic runs showed exactly this.
Paired, interleaved repetitions	Host speed drifted between repetitions (mean slowdown 0.95–1.34), so only differences within the same repetition are reliable.
KVM guest instead of the host	The WSL2 host kernel has no sched_ext. The limitation this imposes on energy measurement is stated in Sections 6.5 and 8.

6. WORK COMPLETED / CURRENT PROGRESS

6.1 Activities Completed So Far

Activity	Outcome	Evidence (repository / commit)
Built the Linux 6.18.53 guest kernel with sched_ext and BTF	Build completed	results/raw/build/kernel_build_3.log; 182a48c
Built scx_compact and the in-tree reference scheduler scx_qmap; first run under KVM	Attach and detach clean, exit 0	results/raw/vm/smoke_1.log; e927c43
Functional test suite F1–F6	12/12 checks pass	results/processed/functional_*.md; e927c43, 958b081
Controller v2: smoothed demand + queue hold	CPUs used under bursty 8×20 % load: 7 → 5 (one run each)	functional F4a; e927c43
Preliminary benchmarks (fixed CPU-bound work, partial load, parameter sweep, context switches)	2–3 repetitions; superseded where the n = 10 run disagrees	results/processed/; e927c43, 43370e2
Latency guard and its diagnostics	Guard implemented; metric corrected over six diagnostic runs	results/raw/vm/guard_diag_1..6.log; 958b081

Activity	Outcome	Evidence (repository / commit)
Repeated validation: 4 conditions × 4 loads × 10 repetitions	160 runs, 0 errors	results/processed/ bench_repeated_20260924T111310Z.md; 20012ec
Bare-metal RAPL energy experiment	Blocked: no RAPL counters in the environment	docs/RAPL_EXPERIMENT.md; c9cbc3e

6.2 Modules Implemented

Module	Implementation status	Size
BPF policy + controller + latency guard	Implemented and validated	473 lines (scx_compact.bpf.c) + 36 (scx_compact.h)
Loader with telemetry and 13 policy options (incl. -n, -L, -P)	Implemented	294 lines (scx_compact.c)
dutywork / latprobe workloads	Implemented	161 / 64 lines
Test, benchmark and analysis scripts	Implemented	tests/
Guest kernel, VM runner, toolchain scripts	Implemented	vm/, scripts/

6.3 Development / Testing Progress

Table 6.3a Functional suite (kernel accounting from /proc/stat; latest run after the latency-guard change)

Check	Result
F1: sched_ext present; reference scheduler scx_qmap attaches and detaches	PASS
F2: scx_compact attaches, runs all phases, exits cleanly (exit 0)	PASS
F3: idle and 1 busy task stay in a 2-CPU set; 4 / 8 busy tasks expand it to 6 / 8; set shrinks back to 2	PASS (5 checks)
F4: under the 8×20 % duty load, compact uses fewer CPUs than EEVDF (5 vs 8); the -n control uses all 8	PASS (2 checks)
F5: a task pinned to CPU 7, outside the 2-CPU set, still runs (CPU 7 100 % busy)	PASS
F6: no sched_ext error or watchdog stall in dmesg	PASS

Evidence excerpt: first run of scx_compact on the sched_ext kernel (results/raw/vm/smoke_1.log). One busy task stays inside the 2-CPU active set:

```
[    6.509431] sched_ext: BPF scheduler "compact" enabled
t=   2.7s active= 2 util= 50% queued=  0 expand=0 shrink=0 busy_cpus(>=5%)= 1 | 101   3   0   0   0   0   0   0
[   13.324692] sched_ext: BPF scheduler "compact" disabled (unregistered from user space)
[smoke] scx_compact exit: 0
```

6.4 Problems Encountered and Adaptations

Problem	Effect	Adaptation	Verified result
WSL2 host kernel has no sched_ext	Scheduler cannot run on the host	Built a 6.18.53 guest kernel; KVM test VM	Guest boots under KVM with sched_ext
C: drive filled during the kernel link	Build failed; WSL file system went read-only	Free-space check during the build, smaller debug information, WSL memory limit	Kernel build completed
In-tree scx_simple calls a kfunc missing in 6.18	Build failure	Set the task's virtual time directly instead; use scx_qmap as the reference scheduler	Scheduler build succeeds
Controller reacted to single 20 ms samples	Bursty load inflated the set (7 of 8 CPUs used)	Smoothed demand + queue hold	5 CPUs used (F4a)
Compaction raised tail latency	Probe p99 +2.9 / +1.4 ms (20 / 40 %)	Latency guard	+0.14 / +0.06 ms (Section 7.4)

Problem	Effect	Adaptation	Verified result
Guard metric counted all waits	Slow wake-ups of idle virtual CPUs (> 1 ms) and per-CPU kernel threads gave ~10 % "late" even with -n	Count only shared-queue waits of tasks allowed on any CPU	-n control: 0 late of ~2,500 dispatches
Wrong denominator (queued dispatches only)	Too few samples; the guard could never act	Denominator = all dispatches of those tasks	~400 samples per second
Rounding in the decay (averaging) step	Late fraction stuck near 10 %	Scaled-integer (fixed-point) arithmetic	Fraction returns to 0 in steady state
Host speed drift between repetitions	Mean slowdown per repetition 0.95–1.34	Interleaved, rotated conditions; paired analysis	Narrow paired intervals
Earlier 3-repetition result: “no latency cost at 20 %”	Wrong conclusion (underpowered, no probe)	n = 10 with latency probe	Revised: clear cost at 20 %
Full-load latency of every sched_ext condition	~2.3 ms higher p99 than EEVDF, also with -n	Documented as a base-policy limitation	Not addressed yet
No RAPL under WSL2/Hyper-V	Energy cannot be measured	No substitute used; documented	Pending native Linux

6.5 Current Project Status

The scheduler, controller and latency guard are implemented and validated in the VM, and feature development is frozen. Reduced CPU usage under partial load and the latency trade-off are **measured** (n = 10, paired). **Energy has not been measured.** The following limitations apply:

- **No RAPL:** the project runs in WSL2 on Hyper-V (detected as wsl). The kernel reports 0 RAPL energy counters, no energy-pkg / energy-cores events are available, the power-capping interface /sys/class/powercap [10] is empty and no MSR device is available. The drivers are present, but the hypervisor does not pass the counters through. A VM inside WSL2 cannot solve this, and the scheduler cannot run natively on the WSL2 kernel.
- **Virtual machine:** an idle vCPU is not a parked physical core, so CPU-usage results show the precondition for energy savings, not the savings themselves. Occasional 100–200 ms latency maxima appear in all conditions, so only p95/p99 are interpreted.
- **Workloads are synthetic** (dutywork, latprobe, stress-ng). The compile and mixed workloads planned in Review-1 are pending.

7. LAST CONTRIBUTIONS / RECENT PROGRESS

7.1 Most Recent Work Completed

- **Latency guard (commit 958b081):** diagnosed the tail-latency cost and implemented and corrected the guard metric over six diagnostic runs. The functional suite still passes 12/12.
- **Repeated validation (commit 20012ec):** 160 runs (4 conditions × 4 loads × 10 repetitions), 0 scheduler, kernel or harness errors, with paired per-repetition analysis.
- **Energy experiment attempt (commit c9cbc3e):** environment check for RAPL; the experiment is blocked and documented, and no estimate was substituted.

7.2 New Functionality or Improvements Added

- **Controller v2:** smoothed demand and a sustained-queue rule replaced reacting to single 20 ms samples. CPUs used under the bursty 8×20 % load fell from 7 to 5 (one run each).
- **Latency guard:** an explainable, configurable safety mechanism (-L, -P) that expands the set only when the scheduler’s own queueing delay exceeds a target, and recovers through the normal hysteresis.
- **Measurement tools:** the dutywork fixed-work workload, the latprobe latency probe, and paired bootstrap statistics.

7.3 Contributions Made Since the Previous Review

Review-1 proposed the design and plan. Since then the team has built the sched_ext environment, implemented scx_compact, validated it functionally, and improved the controller twice (smoothing, then the latency guard). The team also replaced 2–3-repetition measurements with a 10-repetition paired experiment, which overturned one earlier conclusion, and checked the feasibility of RAPL measurement.

7.4 Current Achievements / Results

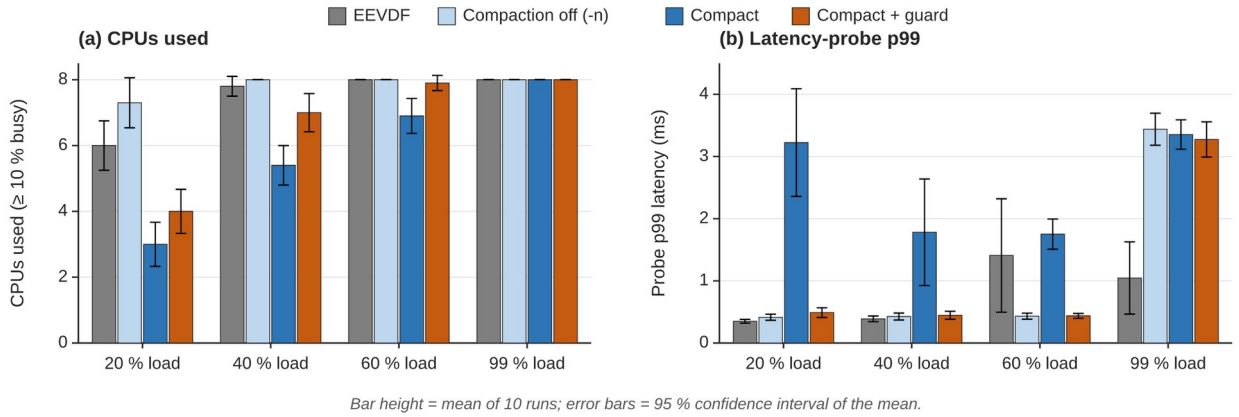


Figure 5. CPUs used and latency-probe p99 per load (n = 10 per bar, mean ± 95 % CI).

Table 7.4a Means over 10 repetitions (MEASURED; results/processed/bench_repeated_20260924T111310Z.md)

Load	CPUs used: EEVDF / -n / compact / guard	CPUs parked (< 1 % busy): EEVDF / compact / guard	Probe p99 (µs): EEVDF / -n / compact / guard
20 %	6.0 / 7.3 / 3.0 / 4.0	1.3 / 5.0 / 3.3	350 / 415 / 3224 / 489
40 %	7.8 / 8.0 / 5.4 / 7.0	0.1 / 2.1 / 0.7	389 / 426 / 1782 / 446
60 %	8.0 / 8.0 / 6.9 / 7.9	0.0 / 1.1 / 0.0	1408 (median 509) / 432 / 1751 / 437
99 %	8.0 / 8.0 / 8.0 / 8.0	0.0 / 0.0 / 0.0	1046 / 3439 / 3354 / 3274

Table 7.4b Paired differences vs EEVDF, mean [95 % bootstrap CI]; bold = interval excludes 0

Load	Compact: CPUs	Compact: probe p99	Guard: CPUs	Guard: probe p99
20 %	**−3.0** [−3.6, −2.3]	**+2.9 ms** [2.2, 3.6]	**−2.0** [−2.8, −1.3]	**+0.14 ms** [0.07, 0.21]
40 %	**−2.4** [−2.9, −1.9]	**+1.4 ms** [0.7, 2.1]	**−0.8** [−1.3, −0.4]	**+0.06 ms** [0.01, 0.12]
60 %	**−1.1** [−1.5, −0.7]	+0.3 ms [−0.4, 1.1]	−0.1 [−0.3, 0.0]	**−1.0 ms** [−1.7, −0.2]
99 %	0.0	**+2.3 ms** [1.7, 2.9]	0.0	**+2.2 ms** [1.6, 2.9]

- **Compaction frees CPUs** at every partial load: 3.0 / 2.4 / 1.1 fewer CPUs at 20 / 40 / 60 %. Completion time shows no significant change, except **+1.6 %** for compact at 20 % [0.1 %, 3.9 %].
- **Compaction has a clear latency cost** at 20 % and 40 % load (probe p99 +2.9 / +1.4 ms). This **revises** the earlier 3-repetition statement that there was no latency cost at 20 %; that statement is withdrawn.
- **The latency guard removes most of that cost:** probe p99 is within +0.14 / +0.06 ms of EEVDF at 20 / 40 %, while 2.0 / 0.8 CPUs stay unused. At 60 % almost no compaction remains (−0.1 CPUs, not significant).
- **At ~100 % load** all sched_ext conditions, **including the compaction-off control**, show about 2.3 ms higher p99 than EEVDF (run-queue wait 425–496 µs vs 16 µs per slice). This is a limitation of the base policy: a woken task can wait for the current 5 ms time slice to end instead of preempting it. It is not caused by compaction.

- **Energy: NOT measured.** A model-based idle-time estimate exists in the repository (tests/analyze_partial.py). It rests on assumed coefficients, is labelled as a model estimate only, and is not reported as an energy result.

8. REMAINING WORK & FUTURE PLAN

8.1 Pending Tasks

1. **Bare-metal energy experiment:** boot Linux directly on the laptop (live USB or dual boot) with a sched_ext-capable kernel and readable RAPL counters, then repeat the same n = 10 experiment and record processor energy before and after each run [9] (docs/RAPL_EXPERIMENT.md).
2. **Workloads from the Review-1 plan:** a compilation workload and a mixed interactive workload, both with the latency probe.
3. **Guard and shrink settings:** repeat 2–3 settings with n = 10 and decide whether the guard should be enabled by default.
4. **Full-load latency:** document the base-policy limitation in the final report, or test a small change that lets a woken task preempt the running one, if this is simple.
5. **Optional:** SMT/CPU-order study on bare metal; reference comparison with scx_lavd if its Rust toolchain can be installed.

8.2 Testing / Validation Still Required

Energy (RAPL) validation on bare metal; validation on real application workloads; a confirmatory run of the chosen guard setting; and re-running the functional suite on the bare-metal kernel.

8.3 Updated Feasibility

Aspect	Assessment
Technical	Demonstrated: the scheduler runs on sched_ext, passes 12/12 checks and completed 160 runs without errors. The only blocked item is energy measurement, which needs native Linux with accessible RAPL counters.
Economic	No additional cost: open-source kernel and tools on the existing laptop. The bare-metal run needs only a bootable Linux USB drive.
Social / environmental	Keeping idle CPUs fully idle is the precondition for lower energy use on everyday partially loaded machines. The project has measured that precondition (fewer CPUs used), but **not** an energy reduction, and makes no energy-saving claim until RAPL data exists.

8.4 Expected Final Outcome

A documented, reproducible sched_ext core-compaction scheduler with a latency guard, together with a measured comparison against EEVDF on CPUs used, completion time and latency. If bare-metal access is obtained, it will also include measured RAPL energy per run; otherwise energy stays explicitly pending. The final report and demonstration will show live telemetry of the active set under changing load.

8.5 Timeline / Plan for Completion

Period	Planned work
Week 1 after Review-2	Bare-metal Linux boot; verify sched_ext and RAPL access; run the energy matrix
Week 2	Compilation and mixed workloads; guard/shrink settings (n = 10)
Week 3	Full-load latency note; optional SMT-order / scx_lavd comparison
Week 4	Final report, README, demonstration and presentation

REFERENCES

[1] T. Heo et al., “sched_ext: Implement BPF extensible scheduler class,” Linux kernel patch series and kernel documentation (Documentation/scheduler/sched-ext.rst).

[2] Linux Kernel Documentation, “EEVDF Scheduler,” kernel scheduler documentation.

[3] Linux Kernel Documentation, “Energy Aware Scheduling,” kernel scheduler documentation.

[4] sched-ext/scx contributors, “scx_lavd,” sched_ext scheduler documentation and source tree, 2025-2026.

[5] Vellore Institute of Technology, “BCSE303L Operating Systems,” Course Outcomes: CO2 – Design scheduling algorithms to compute and compare various scheduling criteria; syllabus version 1.0.

[6] I. Stoica and H. Abdel-Wahab, “Earliest Eligible Virtual Deadline First: A Flexible and Accurate Mechanism for Proportional Share Resource Allocation,” Technical Report TR-95-22, Department of Computer Science, Old Dominion University, 1995.

[7] E. Le Sueur and G. Heiser, “Dynamic Voltage and Frequency Scaling: The Laws of Diminishing Returns,” in Proc. Workshop on Power Aware Computing and Systems (HotPower ’10), USENIX, 2010.

[8] H. David, E. Gorbatov, U. R. Hanebutte, R. Khanna, and C. Le, “RAPL: Memory Power Estimation and Capping,” in Proc. ACM/IEEE Int. Symp. on Low Power Electronics and Design (ISLPED ’10), 2010, pp. 189–194. doi:10.1145/1840845.1840883.

[9] K. N. Khan, M. Hirki, T. Niemi, J. K. Nurminen, and Z. Ou, “RAPL in Action: Experiences in Using RAPL for Power Measurements,” ACM Trans. Modeling and Performance Evaluation of Computing Systems, vol. 3, no. 2, Art. 9, 2018. doi:10.1145/3177754.

[10] Linux Kernel Documentation, “Power Capping Framework” (Documentation/power/powercap/powercap.rst).